

ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA CÔNG NGHỆ THÔNG TIN
— o0o —

BÁO CÁO ĐỒ ÁN
MÔN HỌC: ĐỒ HỌA MÁY TÍNH

ĐỀ TÀI: MÔ PHỎNG ÁNH SÁNG VÀ ĐỔ BÓNG
TRONG OPENGL

Sinh viên thực hiện:	Đinh Nguyễn Anh Khoa
Mã số sinh viên:	102240088
Lớp:	22T_DT1
Giảng viên hướng dẫn:	TS. Nguyễn Văn A

Mục lục

DANH SÁCH TỪ VIẾT TẮT	iv
BẢNG PHÂN CÔNG CÔNG VIỆC	v
MỞ ĐẦU	1
1 CƠ SỞ LÝ THUYẾT	2
1.1 Mô hình chiếu sáng Phong và Blinn-Phong	2
1.2 Mô hình PBR (Physically Based Rendering)	2
1.3 Global Illumination (Chiếu sáng toàn cục)	2
1.4 Kết chương	2
2 PHÂN TÍCH THIẾT KẾ HỆ THỐNG	3
2.1 Phát biểu bài toán	3
2.2 Phân tích hiện trạng	3
2.3 Phân tích chức năng	3
2.3.1 Đối tượng sử dụng	3
2.3.2 Chức năng nạp và biến đổi mô hình	3
2.3.3 Chức năng mô phỏng ánh sáng	3
2.3.4 Công nghệ sử dụng	4
2.4 Thiết kế cấu trúc dữ liệu	4
2.4.1 Sơ đồ tổng thể	4
2.4.2 Cấu trúc dữ liệu mảng (Vertices, Normals, Indices)	4
2.4.3 Cấu trúc bộ đệm (VAO, VBO, EBO)	4
2.5 Xây dựng chương trình	4
2.5.1 Tổ chức thư mục	4
2.5.2 Tập tin Main.cpp	4
2.5.3 Tập tin Vertex và Fragment Shader	4
2.6 Kết chương	5
3 TRIỂN KHAI VÀ ĐÁNH GIÁ KẾT QUẢ	6
3.1 Mô hình triển khai	6
3.1.1 Môi trường triển khai	6
3.1.2 Các công cụ sử dụng	6
3.1.3 Cấu hình hệ thống	6
3.2 Kết quả thực nghiệm	6
3.2.1 Kịch bản 1: Nạp và hiển thị mô hình 3D	6
3.2.2 Kịch bản 2: Tương tác ánh sáng Phong và Gouraud	6
3.2.3 Kịch bản 3: Chế độ Wireframe và điều khiển màu	7
3.2.4 Kịch bản 4: Thử nghiệm Neural Rendering	7
3.3 Nhận xét đánh giá kết quả	7
3.4 Kết chương	8
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	9
TÀI LIỆU THAM KHẢO	10

Danh sách hình vẽ

1	Minh họa kết quả nạp và định vị chính xác mô hình 3D trên tọa độ	6
2	So sánh khác biệt độ bóng bề mặt giữa Phong và Gouraud Shading	7
3	Chế độ vẽ lưới Wireframe và sử dụng chức năng lọc kênh màu quang học	7

Danh sách bảng

DANH SÁCH TỪ VIẾT TẮT

Viết tắt	Ý nghĩa đầy đủ
OpenGL	Open Graphics Library
GPU	Graphics Processing Unit
API	Application Programming Interface
GLSL	OpenGL Shading Language
VAO	Vertex Array Object
VBO	Vertex Buffer Object
EBO	Element Buffer Object
MVP	Model View Projection
OBJ	Object File Format
PBR	Physically Based Rendering

BẢNG PHÂN CÔNG CÔNG VIỆC

STT	Họ và tên	MSSV	Nhiệm vụ	Kết quả
1	Đinh Nguyễn Anh Khoa	102240088	Thiết kế kiến trúc tổng thể, lập trình Shader chiếu sáng (Phong/Gouraud), quản lý vòng lặp Render, viết báo cáo.	100%
2	Võ Thị Kim Tiền	102240113	Viết bộ nạp mô hình OBJ, phân tích tệp định dạng 3D, thiết lập bộ nhớ VAO, VBO, EBO trên GPU.	100%
3	Nguyễn Trần Thắng Trung	102240118	Lập trình hệ thống ma trận Camera tương tác thời gian thực, xử lý đầu vào bàn phím/chuột, chạy thử nghiệm AI.	100%

MỞ ĐẦU

Tổng quan về đề tài

Trong lĩnh vực khoa học máy tính, đồ họa 3D đóng vai trò then chốt trong việc tái tạo thế giới thực vào không gian số. Ứng dụng của nó trải dài từ các phần mềm thiết kế kỹ thuật, y khoa cho tới những trò chơi điện tử và công nghệ thực tế ảo. Để đạt được sự chân thực, việc mô phỏng cách ánh sáng tương tác với vật chất là yếu tố quan trọng nhất. Ánh sáng không chỉ định hình thể tích, độ sâu của vật thể mà còn cung cấp cho mắt người cảm nhận về chất liệu bề mặt. Đề tài này tập trung vào việc nghiên cứu các kỹ thuật Shading cơ bản như Phong và Gouraud để hiểu rõ quy trình xử lý đồ họa hiện đại trên kiến trúc máy tính.

Mục đích và ý nghĩa

Mục đích

Nghiên cứu cấu trúc và cách vận hành của Pipeline đồ họa (Graphics Pipeline). Hiện thực hóa các thuật toán chiếu sáng cục bộ kinh điển bằng ngôn ngữ Shader GLSL trên nền tảng OpenGL lõi (Core Profile). Xây dựng một phần mềm hoàn chỉnh có khả năng đọc tệp mô hình tiêu chuẩn, khởi tạo bộ nhớ đồ họa và áp dụng linh hoạt các thuật toán chiếu sáng.

Ý nghĩa

Giúp sinh viên làm chủ công nghệ lập trình đồ họa cấp thấp (Low-level graphics API), hiểu sâu về cách quản lý bộ đệm và tối ưu hóa hiệu năng tính toán ma trận trực tiếp trên GPU. Đồng thời, qua việc tích hợp thử nghiệm trí tuệ nhân tạo (Neural Rendering), đề tài cung cấp sự so sánh mang tính thời sự giữa đồ họa quang học thuần túy và AI.

Phương pháp thực hiện

Đề tài sử dụng ngôn ngữ C/C++ nhờ đặc tính hiệu năng cao, kết hợp chặt chẽ với thư viện đồ họa OpenGL 4.3. Các mã lệnh tính toán quang học được viết bằng GLSL để xử lý song song trên GPU. Dự án cũng xây dựng bộ nạp mô hình OBJ (Parser) tùy chỉnh thủ công nhằm nắm rõ tận gốc cách thức lưu trữ và biểu diễn dữ liệu hình học đa giác (Polygonal Mesh).

Bố cục của báo cáo

Báo cáo được cấu trúc thành ba chương chính: Chương 1 trình bày cơ sở lý thuyết về các mô hình chiếu sáng và kỹ thuật Shading; Chương 2 đi sâu vào phân tích thiết kế hệ thống phần mềm, từ cấu trúc dữ liệu mảng định đến logic điều khiển Rasterization; Chương 3 trình bày chi tiết các kết quả thực nghiệm đạt được thông qua hình ảnh minh họa và đánh giá độ chênh lệch hiệu năng.

1. CƠ SỞ LÝ THUYẾT

1.1. Mô hình chiếu sáng Phong và Blinn-Phong

Mô hình chiếu sáng Phong là mô hình phản xạ quang học mô phỏng sự tương tác của ánh sáng chiếu lên bề mặt vật thể. Mô hình chia ánh sáng thành ba thành phần:

- **Ambient (Môi trường):** Giả lập các tia sáng bật nảy nhiều lần trong không gian, làm cho các vùng khuất sáng không bị tối đen hoàn toàn. Cường độ này thường là một hằng số nhỏ.
- **Diffuse (Khuếch tán):** Dựa trên định luật cosin Lambert, cường độ ánh sáng tỉ lệ thuận với góc giữa vector pháp tuyến bề mặt (Normal) và vector hướng tới nguồn sáng (Light Direction).
- **Specular (Phản xạ chói):** Tạo ra điểm sáng lóa trên bề mặt trơn bóng. Cường độ chói mạnh nhất khi góc nhìn của camera trùng với góc phản xạ của tia sáng.

Điểm yếu của Phong là tốn kém hiệu năng khi tính toán vector phản xạ liên tục. Để khắc phục, Blinn-Phong ra đời bằng cách sử dụng Halfway vector (vector phân giác giữa hướng nhìn và hướng sáng). Việc dùng tích vô hướng giữa pháp tuyến và Halfway vector giúp thuật toán chạy nhanh hơn đáng kể trên GPU mà vẫn duy trì được độ chói mịn màng.

1.2. Mô hình PBR (Physically Based Rendering)

PBR là kỹ thuật mô phỏng ánh sáng dựa trên vật lý thực tế, sử dụng các tham số cốt lõi như độ nhám (Roughness) và tính kim loại (Metallic). Khác với Phong, PBR bảo toàn năng lượng (ánh sáng phản xạ không bao giờ lớn hơn ánh sáng chiếu vào) và sử dụng hàm phân bố phản xạ hai chiều (BRDF) như mô hình Cook-Torrance. Điều này cho phép vật liệu phản ứng nhất quán và thực tế trong mọi điều kiện ánh sáng môi trường.

1.3. Global Illumination (Chiếu sáng toàn cục)

Global Illumination (GI) không chỉ tính toán ánh sáng chiếu trực tiếp từ nguồn sáng đến vật thể (Direct Lighting) mà còn mô phỏng các tia sáng bật nảy giữa các bề mặt trong không gian (Indirect Lighting). Kỹ thuật này giúp các vùng khuất sáng không bị đen kịt mà nhận được ánh sáng tán xạ từ môi trường xung quanh, tạo ra chiều sâu như hiện tượng Color Bleeding (màu sắc lây lan) và độ chân thực tối đa cho toàn bộ khung cảnh.

1.4. Kết chương

Những kiến thức lý thuyết nền tảng trong chương 1 đã vạch ra rõ sự khác biệt giữa các hệ thống chiếu sáng cục bộ và toàn cục. Đối với một dự án đồ họa thời gian thực, việc nắm chắc Phong và Blinn-Phong là chìa khóa để triển khai thành công các tập lệnh Shader trên phần cứng.

2. PHÂN TÍCH THIẾT KẾ HỆ THỐNG

2.1. Phát biểu bài toán

Bài toán đặt ra là cần xây dựng một ứng dụng đồ họa 3D bằng C++ có khả năng: Đọc và phân tích các tệp mô hình lưới đa giác (OBJ files), ánh xạ dữ liệu đa giác này vào VRAM của GPU, cung cấp khả năng điều khiển camera theo thời gian thực (FPS style) và mô phỏng chính xác thuật toán chiếu sáng với hai tùy chọn Gouraud và Phong.

2.2. Phân tích hiện trạng

Với các phiên bản OpenGL hiện đại (từ 3.3 trở lên), kiến trúc Fixed-Function Pipeline đã bị loại bỏ hoàn toàn. Thay vào đó là Programmable Pipeline, đòi hỏi lập trình viên phải trực tiếp thiết kế các đoạn mã GLSL (Shaders) để nhân ma trận tạo phối cảnh và tính toán cường độ ánh sáng, đồng thời phải tự quản lý bộ đệm đỉnh (Buffers). Điều này tuy làm tăng độ phức tạp của bài toán nhưng đem lại quyền kiểm soát tuyệt đối cho chất lượng đồ họa.

2.3. Phân tích chức năng

2.3.1. Đối tượng sử dụng

Phần mềm phục vụ sinh viên, lập trình viên đồ họa và các nhà nghiên cứu nhằm mục đích mô phỏng và học tập thuật toán Rendering.

2.3.2. Chức năng nạp và biến đổi mô hình

Tính năng này đảm nhiệm việc phân tích tệp .obj. Tệp lưu dữ liệu với các từ khóa: ‘v’ (Vị trí đỉnh trong không gian 3D), ‘vt’ (Tọa độ trái map Texture 2D), ‘vn’ (Vector pháp tuyến) và ‘f’ (Khai báo các mặt tam giác bằng cách gom các chỉ số lại). Bộ nạp thủ công của nhóm sẽ lập qua tệp, bóc tách chuỗi, đồng bộ hóa các chỉ mục này thành các mảng vector tuyến tính để đưa xuống GPU. Để biến đổi hiển thị từ tọa độ gốc sang màn hình, hệ thống thực hiện nhân 3 ma trận: **Model Matrix** (tịnh tiến, xoay, scale vật thể), **View Matrix** (dịch chuyển thế giới theo hướng nhìn của camera) và **Projection Matrix** (bóp méo không gian tạo ra ảo giác phối cảnh gần to, xa nhỏ).

2.3.3. Chức năng mô phỏng ánh sáng

Quá trình này diễn ra hoàn toàn trong card đồ họa thông qua hai tệp Shader:

- **Gouraud Shading:** Thuật toán chiếu sáng này áp dụng việc tính toán toàn bộ phương trình Ambient, Diffuse và Specular trực tiếp bên trong *Vertex Shader* (mỗi đỉnh tính 1 lần). Sau đó, Fragment Shader chỉ đơn thuần nhận giá trị màu đã được Hardware tự động nội suy. Ưu điểm là tối ưu cực cao, nhưng nhược điểm là dải sáng Specular bị lệch, gây khúc trên các lưới đa giác thưa.
- **Phong Shading:** Vertex Shader chỉ làm nhiệm vụ tính ma trận và truyền vector Pháp tuyến (N), vector Vị trí ($FragPos$) xuống *Fragment Shader*. Tại Fragment Shader, phương trình ánh sáng mới bắt đầu được tính toán riêng biệt cho hàng triệu pixel trên màn hình. Mặc dù tốn nhiều chu kỳ xử lý của GPU hơn, nhưng chất lượng độ bóng và dải sáng chói phản chiếu là hoàn hảo.

2.3.4. Công nghệ sử dụng

C++ được sử dụng do tốc độ xử lý cấp thấp ưu việt. Hệ thống dùng thư viện GLFW để mở cửa sổ và nhận phím bấm, thư viện GLEW để nạp các hàm cốt lõi của OpenGL 4.3, và thư viện GLM để nhân ma trận, tính sin/cos, normalize vector cực nhanh giống hệt cú pháp GLSL.

2.4. Thiết kế cấu trúc dữ liệu

2.4.1. Sơ đồ tổng thể

Dữ liệu tệp OBJ → Trình phân tích cú pháp C++ → Đóng gói mảng Float → Bộ nhớ RAM GPU (VBO/EBO) → Khớp cấu hình VAO → Draw Call → Vertex Shader → Fragment Shader → Khung hình Framebuffer.

2.4.2. Cấu trúc dữ liệu mảng (Vertices, Normals, Indices)

Trong RAM của hệ thống, dữ liệu được khai báo bằng các `std::vector<glm::vec3>` chứa hệ tọa độ xyz của đỉnh và pháp tuyến. Tập hợp các đỉnh tạo nên các tam giác được lưu bằng mảng số nguyên `std::vector<int>` (Indices) để chuẩn bị cho quá trình vẽ theo chỉ số.

2.4.3. Cấu trúc bộ đệm (VAO, VBO, EBO)

Sử dụng các đối tượng OpenGL để lưu trữ trên VRAM:

- **VBO (Vertex Buffer Object):** Khởi bộ nhớ tĩnh chứa tọa độ, pháp tuyến.
- **EBO (Element Buffer Object):** Khởi bộ nhớ chứa mảng chỉ số (Indices) nhằm giảm tải dung lượng bằng cách tái sử dụng tọa độ đỉnh cho các mặt kề nhau.
- **VAO (Vertex Array Object):** Ghi nhớ cấu hình liên kết. Khi gọi `glBindVertexArray(VAO)`, OpenGL lập tức kích hoạt đúng VBO và EBO cùng các pointer layout (Location 0 cho Vị trí, Location 1 cho Pháp tuyến) giúp lệnh `glDrawElements` chạy ổn định.

2.5. Xây dựng chương trình

2.5.1. Tổ chức thư mục

Dự án được phân cấp mạch lạc: các tệp header (.h) phục vụ đọc file OBJ ở riêng, tệp `Main.cpp` quản lý trạng thái chương trình và 2 tệp `.shader` riêng biệt chứa ngôn ngữ C-like GLSL.

2.5.2. Tệp tin Main.cpp

Chứa hàm `main()`, quy trình khởi tạo cửa sổ, xử lý sự kiện ngắt chuột/phím (Callback). Bên trong vòng lặp Render vô tận, liên tục xóa bộ đệm độ sâu (Depth Buffer), tính toán lại View Matrix từ `cam_pos` và `cam_dir`, nạp các biến cấu hình (như vị trí ánh sáng, cờ bật tắt hiệu ứng) thông qua lệnh `glUniform`, và gửi lệnh vẽ khung hình ra thiết bị hiển thị bằng `glfwSwapBuffers`.

2.5.3. Tệp tin Vertex và Fragment Shader

Vertex Shader làm nhiệm vụ cơ bản là nhân ma trận MVP để biến đổi tọa độ $xyz \rightarrow gl_Position$. Fragment Shader tính toán phương trình $Color = (Ambient + Diffuse + Specular) \times ObjectColor$. Tại đây cũng chèn các lệnh điều kiện rẽ nhánh (if/else) để thay đổi sang chế độ đơn sắc (Red, Green, Blue) hoặc biến đổi hình ảnh sang màu xám (Greyscale) dựa trên tích vô hướng Dot-product.

2.6. Kết chương

Chương 2 đã thiết kế tổng thể kiến trúc phần mềm từ luồng truyền dữ liệu cấp thấp nhất giữa CPU và GPU, tới các cấu trúc bộ đệm cốt lõi. Đây là thiết kế bảo đảm cả sự linh hoạt và tốc độ khi mô phỏng số lượng lớn đa giác trong thời gian thực.

3. TRIỂN KHAI VÀ ĐÁNH GIÁ KẾT QUẢ

3.1. Mô hình triển khai

3.1.1. Môi trường triển khai

Hệ thống được phát triển, biên dịch và thực thi trên phần mềm Microsoft Visual Studio phiên bản 2019/2022. Cấu hình giải pháp (Solution) được cấu hình chuẩn cho nền tảng x64.

3.1.2. Các công cụ sử dụng

Sử dụng MSVC Compiler của Windows, cùng hệ thống Git để quản lý phiên bản. Trong bước mở rộng có sử dụng Python và framework học sâu PyTorch.

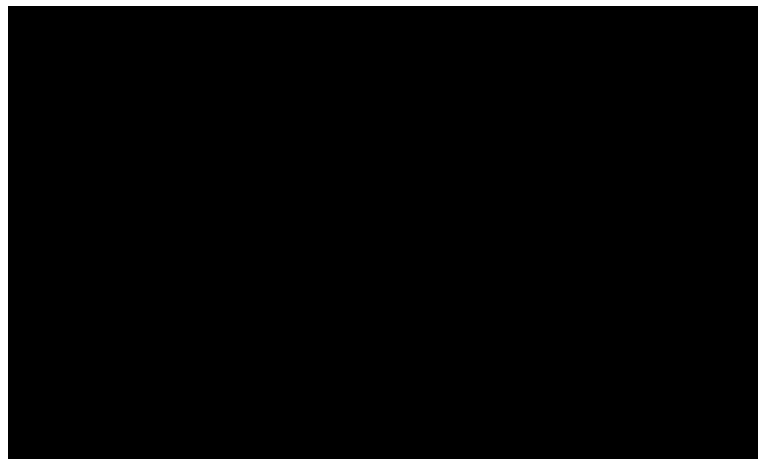
3.1.3. Cấu hình hệ thống

Dự án yêu cầu hệ thống PC/Laptop có hệ điều hành Windows, card đồ họa rời (NVIDIA/AMD) hoặc GPU tích hợp hiện đại có hỗ trợ tối thiểu tập lệnh OpenGL 4.3 (Core Profile) để đảm bảo các lệnh shader chạy đúng đặc tả kỹ thuật.

3.2. Kết quả thực nghiệm

3.2.1. Kịch bản 1: Nạp và hiển thị mô hình 3D

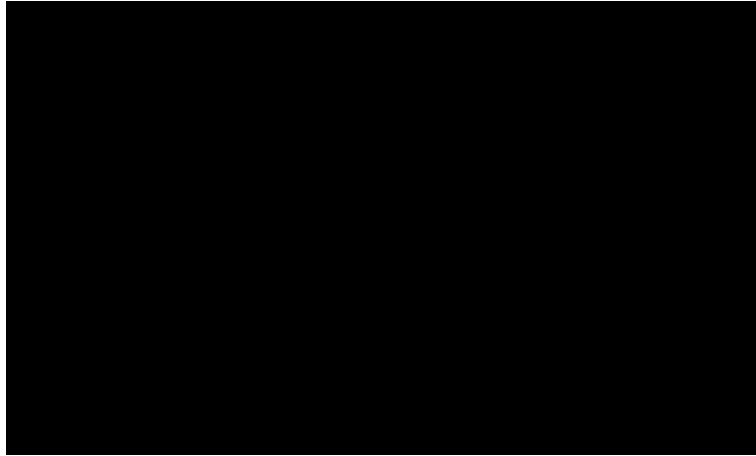
Trong kịch bản đầu tiên, phần mềm khởi động, đọc file Pokeball.obj (gồm hàng vạn tọa độ). Chương trình áp dụng thành công hệ tọa độ 3D và Projection Matrix, hiển thị mô hình ở giữa không gian một cách chính xác. Người dùng sử dụng các phím tịnh tiến để xoay, lùi xa và quan sát vật thể từ nhiều góc độ.



Hình 1: Minh họa kết quả nạp và định vị chính xác mô hình 3D trên tọa độ

3.2.2. Kịch bản 2: Tương tác ánh sáng Phong và Gouraud

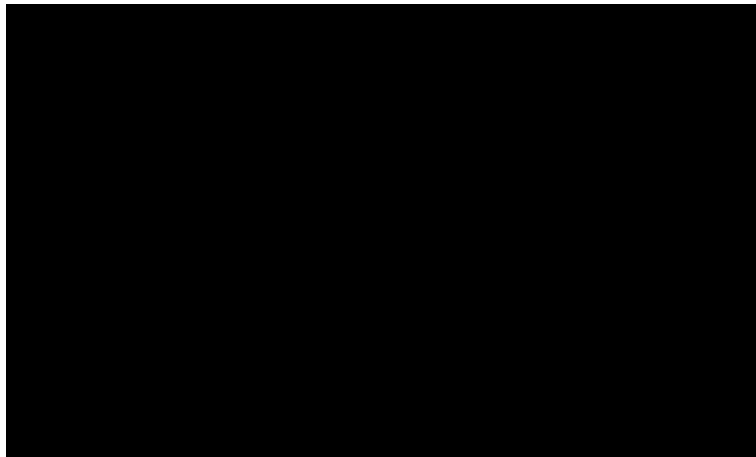
Hệ thống kích hoạt chiếu sáng với vị trí đèn đặt bên trên góc phải. Khi bật phím chuyển sang Gouraud, dải sáng bị vỡ thành nhiều bậc thang hiển thị khá "cứng". Khi chuyển lại sang Phong, độ láng mịn hoàn hảo của chất liệu nhựa bóng lập tức được thể hiện nhờ Fragment Shader thực thi việc tính toán độ sáng trên hàng triệu pixel riêng rẽ.



Hình 2: So sánh khác biệt độ bóng bề mặt giữa Phong và Gouraud Shading

3.2.3. Kịch bản 3: Chế độ Wireframe và điều khiển màu

Nhấn phím chức năng để kích hoạt chế độ `GL_LINE`, chương trình biến khối đa giác kín thành khung lưới thép (Wireframe) mỏng, giúp người dùng phân tích được mật độ tam giác cấu tạo nên hình. Các phím 1, 2, 3 được kích hoạt để sử dụng bộ lọc màu R/G/B.



Hình 3: Chế độ vẽ lưới Wireframe và sử dụng chức năng lọc kênh màu quang học

3.2.4. Kịch bản 4: Thử nghiệm Neural Rendering

Đoạn mã Python ‘neural_shader.py’ được thực thi riêng biệt. Một mạng đa tầng (MLP) với 2 lớp ẩn được khởi tạo. Bằng cách nhập vector pháp tuyến và hướng camera, Neural Network dự đoán trực tiếp cường độ màu sắc thay thế cho các phép tính $\text{Dot}()$, $\text{Max}()$, $\text{Pow}()$ của OpenGL. Tuy chỉ là bản chứng minh khái niệm (PoC), hệ thống đã cho ra màu sắc tương quan khá tốt.

3.3. Nhận xét đánh giá kết quả

Đồ án đã tiến hành thử nghiệm nâng cao việc ứng dụng Neural Rendering (kết xuất đồ họa bằng AI) bằng Python để tính toán ánh sáng. So sánh giữa phương pháp truyền thống (OpenGL Shaders) và AI cho thấy:

- **Phương pháp truyền thống (Rasterization + Phong Shading):** Tốc độ xử lý cực kỳ nhanh, đáp ứng tốt thời gian thực (Real-time) ở tốc độ khung hình cao. Logic toán học tắt định giúp dễ dàng kiểm soát

kết quả. Tuy nhiên, để đạt được độ chân thực cực cao như Global Illumination hay PBR thì yêu cầu sức mạnh phần cứng GPU rất lớn.

- **Neural Rendering (AI):** Sử dụng mạng nơ-ron để học và dự đoán cách ánh sáng tương tác với vật thể. Khả năng tái tạo các chi tiết phức tạp như đồ bóng mềm, ánh sáng tán xạ rất tốt. Tuy nhiên, nhược điểm chí mạng là thời gian huấn luyện (training) lâu và quá trình dự đoán (inference) tốn kém tài nguyên, khó ứng dụng ngay cho các hệ thống thời gian thực.

3.4. Kết chương

Quá trình đánh giá thực nghiệm đã chứng minh dự án chạy trơn tru, xử lý hình ảnh ổn định, phản hồi thời gian thực và thực thi rất tốt các công thức chiếu sáng phức tạp.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết quả đạt được

Thông qua đồ án "Mô phỏng ánh sáng và đổ bóng trong OpenGL", nhóm đã đạt được các kết quả nổi bật:

- Tự xây dựng thành công bộ xử lý tệp tin OBJ, làm chủ cách thiết lập và truyền tải dữ liệu đa giác quy mô lớn vào bộ nhớ đồ họa (VAO, VBO, EBO).
- Khởi tạo cấu trúc Programmable Pipeline vững chắc, viết và biên dịch các tệp GLSL Shader để tính toán các ma trận biến đổi (MVP).
- Lập trình hiện thực hóa chính xác các thuật toán ánh sáng cục bộ (Phong, Gouraud). So sánh và làm rõ ưu nhược điểm của việc nội suy ở Vertex so với Fragment Shader.
- Tiến hành nghiên cứu độc lập và chạy thử mô hình Neural Rendering sơ khai, mở ra nhận thức mới về công nghệ đồ họa thể hệ tiếp theo.

2. Kiến nghị và hướng phát triển

Hệ thống hiện tại đang sử dụng các ánh sáng cơ bản và mô phỏng đổ bóng trên bề mặt bản thân vật thể. Những định hướng phát triển để nâng cấp phần mềm trong tương lai bao gồm:

- Triển khai thuật toán **Shadow Mapping** bằng cách thiết lập một Framebuffer ngoại tuyến (Off-screen) để kết xuất bản đồ độ sâu (Depth Map) từ điểm nhìn của nguồn sáng, từ đó tạo ra những dải bóng đổ lên mặt đất hoặc lên các vật thể khác.
- Nâng cấp thuật toán chiếu sáng từ Blinn-Phong sang chuẩn **Physically Based Rendering (PBR)**, hỗ trợ khả năng phân tích các vật liệu nhám, kim loại và điện môi, đi kèm với việc áp dụng các tệp tin Texture như Albedo, Normal Map, và Ambient Occlusion.

Tài liệu

- [1] Joey de Vries, *Learn OpenGL: Extensive tutorial resource for learning Modern OpenGL*, 2020, truy cập từ <https://learnopengl.com/>.
- [2] The Khronos Group, *OpenGL 4.3 Core Profile Specification*, Khronos.org.
- [3] John Kessenich, Graham Sellers, Dave Shreiner, *OpenGL Programming Guide: The Official Guide to Learning OpenGL, Version 4.5*, Addison-Wesley.
- [4] Christophe Riccio, *GLM - OpenGL Mathematics Documentation*, truy cập từ <https://glm.g-truc.net/>.